

BỘ THÔNG TIN VÀ TRUYỀN THÔNG
HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
-----o0o-----
INFORMATION SYSTEM ANALYSIS AND DESIGN



ASSIGNMENT 06

Giảng viên hướng dẫn : Trần Đình Quế
Họ và tên sinh viên : Nguyễn Tiến Đạt
Mã sinh viên : B23DCCE017
Lớp : E23TTNT02

Hà Nội – 2026

Contents

1. Introduction.....	3
2. Problem Description.....	3
3. Requirements Analysis.....	4
3.1. Actor.....	4
3.2. Functional Requirements	4

3.3. Non-Functional Requirements	4
3.4. Inputs and Outputs	4
4. Use Case Model	5
4.1. Use Case Descriptions	5
4.2. Use Case Relationships.....	5
4.3. Main Scenario: Search by Voice	6
5. Three-Layer Architecture	6
5.1. Presentation Layer	7
5.2. Application / Intelligence Layer.....	7
5.3. Data Layer	7
5.4. Dependencies and Processing.....	7
6. UML Sequence Diagram.....	8
6.1. Participants	9
6.2. Interaction Sequence.....	9
6.3. Alternative Flows.....	9
7. Python Implementation	11
7.1. Project Structure	11
7.2. Product Repository and Dataset.....	11
7.3. Search and Ranking Services.....	11
7.4. Running the Program.....	11
8. Multimodal Search Method.....	12
8.1. Text Search	12
8.2. Voice Search	12
8.3. Image Similarity Search.....	12
8.4. Result Presentation	12
9. Experimental Results	12
9.1. Test Set and Overall Results	12
9.2. Image Model Comparison	13
9.3. Demonstration Queries	13
9.4. Verification	15
10. Discussion	15
10.1. Strengths.....	15
10.2. Limitations.....	15
10.3. Recommended Improvements	15
11. Conclusion.....	15
11.1. References	16

1. Introduction

This report presents the analysis, design, and implementation of a prototype multimodal search system for e-commerce. The system supports product searches through text, browser microphone input, simulated voice transcripts for repeatable CLI demonstrations, and uploaded product images. These inputs follow a unified process: query representation, candidate retrieval, ranking, and result presentation.

The system adopts a three-layer architecture: the Presentation Layer handles user interaction, the Application / Intelligence Layer processes queries and ranks results, and the Data Layer manages product information, images, and feature vectors. UML diagrams created in Visual Paradigm guide the Python implementation.

The final prototype combines deterministic keyword matching with real browser microphone input and pretrained visual embeddings. Text and recognized speech share bilingual Vietnamese-English normalization. Image search uses CLIP ViT-B/32 to encode uploaded images and catalog photographs, then ranks products by cosine similarity. The implementation therefore covers the required simulated voice workflow in the CLI and extends it with microphone recognition and arbitrary image upload in the web interface.

2. Problem Description

Customers may search for products using keywords, spoken requests, or reference images. These inputs require different processing methods but should produce a consistent output: a ranked list of relevant products.

The system must support three search scenarios:

- **Text search:** A customer enters a query such as “black running shoes.” The system matches query terms against product information.
- **Voice search:** On the web, the customer records a request through the microphone and the browser converts speech to a transcript. The transcript remains editable and then follows the same normalization and retrieval process as text search. The CLI retains the assignment's simulated transcript workflow for repeatable demonstration.
- **Image search:** A customer uploads a JPEG, PNG, or WebP image. A pretrained CLIP ViT-B/32 vision encoder converts its pixels into a 512-dimensional semantic embedding. The system compares this vector with embeddings inferred from the catalog photographs and returns the nearest products.

The product repository contains 40 products across four balanced categories: shoes, bags and backpacks, clothing, and drinkware. Each category contains ten products. Every record has a stable identifier, English and Vietnamese names, category, color, price, stock quantity, description, and a unique credited photograph. Search results display product details, rank, and score. Empty input, invalid uploads, unsupported formats, and no-match queries receive clear feedback.

The broader system also includes order lookup and order details, while the core implementation focuses on product search. The main design challenge is to integrate different input methods into a shared search workflow while maintaining clear separation between user interaction, application logic, and data access.

3. Requirements Analysis

The requirements define the system's users, supported functions, quality constraints, and expected inputs and outputs. The core prototype focuses on product search, while order search is included in the broader system design.

3.1. Actor

Customer is the primary actor. Customers search by text, microphone, or uploaded image and inspect ranked results. The CLI also supports the required simulated voice transcript. In the extended design scope, customers can search for and view orders.

3.2. Functional Requirements

ID	Requirement	Scope
FR01	The system shall allow customers to search products using text keywords.	Core
FR02	The system shall accept a supplied transcript for repeatable voice-search demonstration and a browser-recognized microphone transcript in the web interface.	Core
FR03	The system shall accept an arbitrary JPEG, PNG, or WebP image and perform similarity search using pretrained CLIP embeddings.	Core
FR04	The system shall convert supported inputs into a common query structure.	Core
FR05	The system shall validate inputs, uploaded file type and size, and return clear feedback for invalid requests or no matches.	Core
FR06	The system shall retrieve candidate products and rank them by a deterministic relevance or cosine-similarity score.	Core
FR07	The system shall provide appropriate feedback for empty queries, unsupported images, and searches with no results.	Core
FR08	The system shall allow customers to view details of a selected product.	Core
FR09	The system shall allow customers to find their orders by ID and view order details.	Extension

3.3. Non-Functional Requirements

ID	Requirement
NFR01 — Maintainability	The system shall separate presentation, application logic, and data access. The presentation layer shall not access stored data directly.
NFR02 — Extensibility	Speech recognition and image encoders shall remain replaceable behind stable service interfaces.
NFR03 — Reliability	Invalid inputs shall produce clear feedback without terminating the application unexpectedly.
NFR04 — Reproducibility	Identical inputs and model files shall produce deterministic ranking, including a defined product-ID tie rule.
NFR05 — Usability	The interface shall identify browser speech processing, local image inference, accepted upload formats, and prototype limitations.
NFR06 — Portability	Uploaded images shall be processed in memory, limited to 8 MB, and never stored by the server.

3.4. Inputs and Outputs

Search Mode	Prototype Input	Processing	Output
Text	Keyword string	Normalization and keyword matching	Ranked products with text-matching scores
Voice	Microphone audio or supplied transcript	Browser speech recognition, transcript validation, then bilingual text search	Ranked products with text-matching scores
Image	JPEG, PNG, WebP upload or demo image	CLIP ViT-B/32 inference and cosine similarity	Ranked products with similarity scores

The dataset contains 40 products with consistent identifiers, balanced category counts, unique image paths, and credited source URLs. Catalog and query embeddings have 512 dimensions and are produced by the same CLIP preprocessing pipeline. The interface identifies browser speech recognition as an external service and reports the image model used for retrieval.

4. Use Case Model

The use case model describes the system's functions from the customer's perspective. The system boundary is named **Multimodal E-Commerce Search System**, with **Customer** as the primary actor.

4.1. Use Case Descriptions

Use Case	Description
Search Product	Find relevant products using a supported input method.
Search by Keyword	Find products by matching text keywords against product information.
Search by Voice	Find products using a microphone transcript or a supplied transcript for the CLI demonstration.
Search by Image	Find visually similar products from an arbitrary supported image upload using CLIP embeddings.
Search Order	Find an order belonging to the customer using its identifier.
View Product	Display details of a selected product.
View Order	Display details of a selected order, including its status and total.

4.2. Use Case Relationships

Search by Keyword, Search by Voice, and Search by Image specialize Search Product through generalization relationships. The web realization accepts real microphone input and arbitrary image uploads, while the CLI preserves deterministic text, simulated voice, and image demonstrations.

The Customer is associated with **Search Product**, **Search Order**, **View Product**, and **View Order**. Product and order viewing are modeled as separate user goals. Order-related use cases represent the extended system scope.

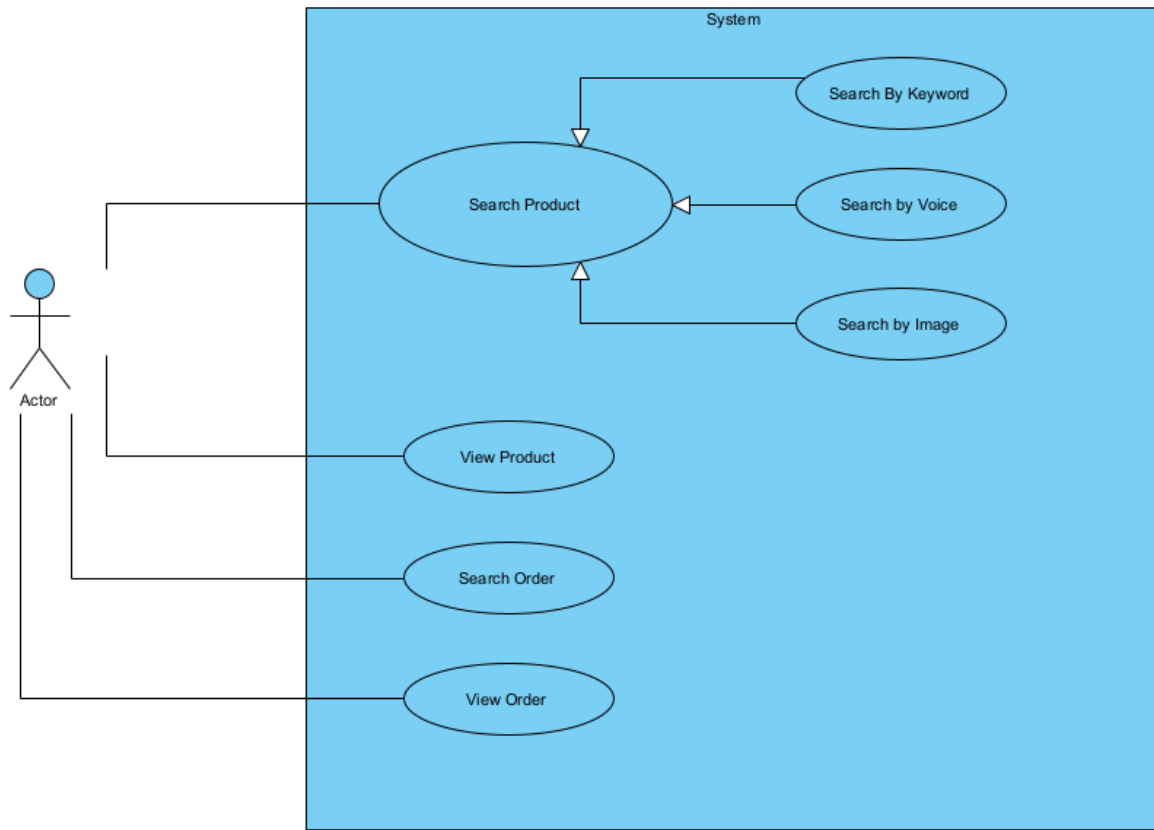


Figure 1. Use case model of the multimodal e-commerce search system.

4.3. Main Scenario: Search by Voice

Item	Description
Primary actor	Customer
Precondition	Product data is available.
Trigger	The customer selects voice search.
Main flow	1. The customer records speech with the web microphone or supplies a CLI transcript. 2. The browser or CLI produces a transcript. 3. The system constructs a query. 4. Candidate products are retrieved and ranked. 5. Results and scores are displayed.
Alternative flows	An empty transcript produces a validation message. A query with no matches produces a no-results message.
Postcondition	Ranked results or an appropriate message are displayed.
Prototype constraint	The web uses browser microphone recognition; the CLI uses a supplied transcript so the required simulated demonstration is repeatable.

5. Three-Layer Architecture

The system follows a three-layer architecture that separates user interaction, query and ranking logic, and data access. This separation allowed the initial artificial image vectors to be replaced by CLIP without changing the common query schema or result-ranking interface.

5.1. Presentation Layer

The Presentation Layer collects customer input and displays search results. It delegates query processing to the Application Layer and does not directly access stored data.

Component	Responsibility
SearchUI	Coordinate user interaction and submit search requests.
VoiceInput	Capture microphone input in the browser or accept a supplied CLI transcript.
ImageUpload	Accept a JPEG, PNG, or WebP upload of at most 8 MB, or select a registered demo image.
SearchResultView	Display product information, ranking scores, and feedback messages.

5.2. Application / Intelligence Layer

The Application Layer interprets inputs, retrieves candidate products, and ranks results.

Component	Responsibility
QueryService	Normalize input and construct a common query object.
SpeechService	Validate the transcript supplied by the browser or CLI before query normalization.
ImageService	Use CLIPImageEncoder to infer a normalized 512-dimensional vector from image pixels.
SearchService	Retrieve text candidates or compare image vectors against the complete catalog index.
RankingService	Sort candidates by score and select the top-k results.

Retrieval and ranking are separate operations: retrieval identifies candidates and computes relevance scores, while ranking determines their final order.

5.3. Data Layer

The Data Layer provides access to product information, order information, images, and feature vectors.

Component	Responsibility
ProductRepository	Provide product retrieval operations.
OrderRepository	Provide order lookup operations in the extended scope.
VectorIndex	Store normalized CLIP vectors for all 40 catalog products during the application process.
ProductDatabase	Store product records, implemented using JSON in the prototype.
OrderDatabase	Store order records in the extended design scope; this component is not implemented in the search prototype.
ImageStorage	Store product images as unique local PNG files with source and license metadata; the CLIP ONNX model is loaded as a separate runtime artifact.

5.4. Dependencies and Processing

Dependencies follow the direction **Presentation** → **Application** → **Data**, with additional dependencies between components within the same layer.

SearchUI and the FastAPI presentation adapter collect input, invoke application services, and render results. QueryService normalizes text and constructs the common query object. SearchService retrieves candidates from ProductRepository and uses VectorIndex plus RankingService for scored output. For image requests, ImageService calls CLIPImageEncoder; catalog images are indexed at startup and uploaded images are processed in memory.

The diagrams represent logical responsibilities. In the final Python implementation, the image-storage responsibility is realized by local image files, while CLIPImageEncoder performs feature extraction. Order components remain an extended design scope and are not part of the product-search prototype.

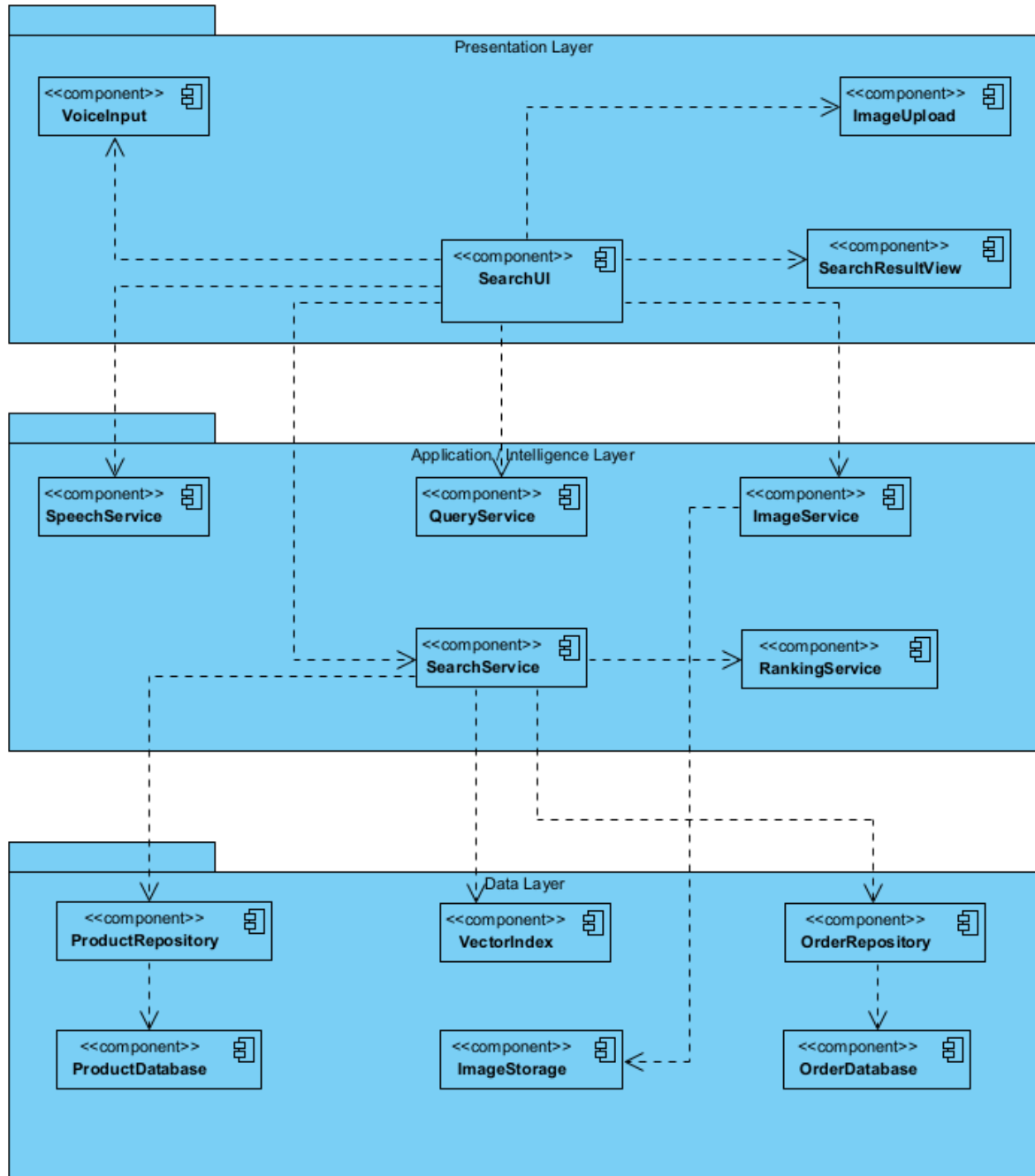


Figure 2. Three-layer architecture of the multimodal e-commerce search system.

6. UML Sequence Diagram

The sequence diagram documents the assignment's required simulated voice scenario. SearchUI receives a supplied transcript, validates it through SpeechService, constructs a normalized query, retrieves candidates, ranks them, and displays the result. The web interface adds microphone recognition before this sequence and sends only the resulting transcript to the Python API.

6.1. Participants

Participant	Responsibility
Customer	Initiate voice search and provide input.
SearchUI	Coordinate user interaction and service calls.
SpeechService	Validate and return the supplied transcript in the deterministic CLI sequence.
QueryService	Normalize the transcript and construct a query object.
SearchService	Retrieve candidate products and calculate matching scores.
ProductRepository	Provide product records.
RankingService	Sort candidates and select the top-k results.

For readability, input collection and result presentation are represented within SearchUI. Their presentation-layer helper components are omitted from this interaction view.

6.2. Interaction Sequence

Step	Interaction	Description
1	Customer → SearchUI: submitVoice(voiceInput)	The customer submits a simulated voice input.
2	SearchUI → SpeechService: transcribe(voiceInput)	SearchUI requests a transcript.
3	SpeechService → SearchUI: return transcript	SpeechService returns the supplied text as the simulated transcription.
4	SearchUI → QueryService: voice_query(transcript)	SearchUI requests a normalized voice-query object.
5	QueryService → SearchUI: return query	QueryService returns the constructed query.
6	SearchUI → SearchService: search(query, top_k)	SearchUI submits the query and requested result limit.
7	SearchService → ProductRepository: all_products()	SearchService requests the product collection.
8	ProductRepository → SearchService: return products	The repository returns the available product records.
9	SearchService → SearchService: retrieve_text_candidates(query, products)	SearchService identifies matching products and calculates relevance scores.
10	SearchService → RankingService: rank(candidates, top_k)	SearchService requests candidate sorting and top-k selection.
11	RankingService → SearchService: return ranked_results	RankingService returns the highest-ranked candidates.
12	SearchService → SearchUI: return ranked_results	SearchService forwards the final results to the interface.
13	SearchUI → SearchUI: display_results(ranked_results)	SearchUI performs an internal operation to display the ranked products and their scores.

The diagram represents the successful search scenario. The final interaction denotes user-visible output rather than a method executed by the Customer.

Synchronous calls are represented by solid message arrows, while return messages use dashed arrows. Activation bars indicate periods of execution. Candidate retrieval is shown as a self-call within SearchService.

6.3. Alternative Flows

An alt combined fragment distinguishes valid and empty transcripts. If the transcript is empty, SearchUI displays a validation message and does not invoke the search workflow.

After processing a valid query, another alt fragment distinguishes non-empty results from an empty result list. The system either displays ranked products or informs the customer that no matching products were found.

The interaction preserves the three-layer architecture: SearchUI invokes application services, and SearchService accesses product data through ProductRepository. Retrieval and ranking remain separate responsibilities.

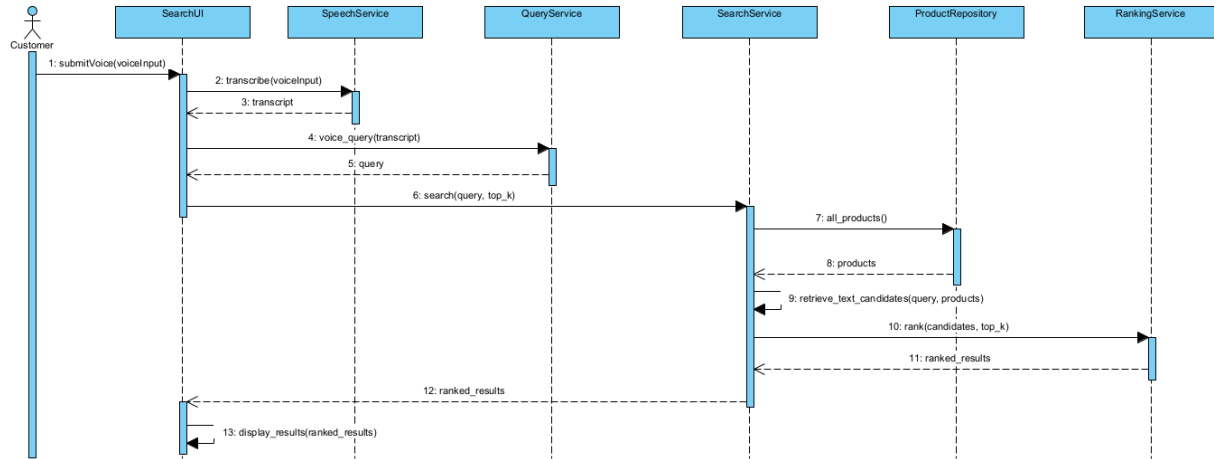


Figure 3. Sequence diagram for product search using simulated voice input.

7. Python Implementation

The implementation uses Python 3.13 and preserves the three-layer design. The CLI composition root and the FastAPI web adapter build the same application services, so both interfaces execute the same repository, query, retrieval, and ranking logic. The web adds HTML, CSS, and JavaScript for microphone control, image upload, validation messages, and bilingual presentation.

7.1. Project Structure

Package or file	Implementation responsibility
data	ProductRepository, VectorIndex, and the ONNX image encoder.
application	Query normalization, speech transcript validation, image encoding, candidate retrieval, cosine similarity, and ranking.
presentation	CLI input coordination and formatted demonstration output.
web	FastAPI endpoints and the browser interface for text, microphone, image upload, catalog, and diagnostics.
datasets	Forty product records, unique product photographs, demo query images, evaluation queries, and photo-source metadata.
models	CLIP ViT-B/32 ONNX INT8 vision encoder and the retained MobileNetV2 comparison model.
tests and scripts	Unit, API, browser, catalog-photo, retrieval, and reproducible evaluation checks.

7.2. Product Repository and Dataset

ProductRepository loads and validates datasets/products.json. The catalog contains 40 products, with ten records in each of four categories. Every product has English and Vietnamese names and a distinct local photograph. photo_sources.json records a separate source URL for each image. The repository returns copies of records so presentation code cannot mutate stored data.

7.3. Search and Ranking Services

QueryService creates a common query object with type, normalized text or embedding, and filters. SearchService selects the retrieval path by query type. RankingService validates top_k, sorts by descending score, uses ascending product ID for deterministic ties, and returns only the requested number of products.

The FastAPI adapter exposes catalog, product-detail, text-search, voice-search, and image-search endpoints. Image uploads are restricted to JPEG, PNG, and WebP, limited to 8 MB, decoded before inference, and processed without permanent storage. The API hides internal paths and returns controlled 4xx or 5xx responses instead of private tracebacks.

7.4. Running the Program

Purpose	Command
Install dependencies	<code>.\venv-web\Scripts\python.exe -m pip install -r requirements-web.txt</code>
Run CLI demonstration	<code>.\venv-web\Scripts\python.exe main.py --demo</code>
Run web application	<code>.\venv-web\Scripts\python.exe -m uvicorn web.app:app --host 127.0.0.1 --port 8011</code>
Run automated tests	<code>.\venv-web\Scripts\python.exe -m unittest discover -s tests -v</code>
Compare image models	<code>.\venv-web\Scripts\python.exe scripts/evaluate_image_models.py</code>

8. Multimodal Search Method

8.1. Text Search

Text input is converted to lowercase, Unicode accents are removed, and configured Vietnamese phrases are mapped to English catalog terms. Longer phrases such as giày chạy bộ and xanh lá are processed before individual tokens. The retrieval score is the number of distinct normalized query tokens found in the product name, category, color, or description. A product is retained when at least one token matches.

8.2. Voice Search

The browser uses SpeechRecognition or webkitSpeechRecognition with vi-VN or en-US. It displays interim text, allows the customer to stop recording, and starts a search when final recognition completes. Only the transcript reaches the Python API; the server does not receive or store microphone audio. The CLI accepts a supplied transcript to reproduce the simulated voice requirement without depending on a microphone or online recognition service.

8.3. Image Similarity Search

The image pipeline center-crops RGB input to 224 by 224 pixels and applies the mean and standard deviation required by CLIP. The quantized CLIP ViT-B/32 vision encoder produces a 512-dimensional semantic vector. The vector is normalized to unit length. Catalog images pass through the same preprocessing and inference path when services start.

For a query vector q and product vector p , cosine similarity is $\text{score}(q, p) = (q \cdot p) / (\|q\| \|p\|)$. Because both vectors are normalized, the score is their dot product. The search compares the query with all 40 indexed products, then RankingService sorts the candidates and applies top_k. CLIP replaced MobileNetV2 class logits because semantic embeddings grouped the product categories more consistently.

8.4. Result Presentation

Every response preserves the original input and reports its mode, processing method, result count, backend duration, product data, rank, and score. Text and voice scores count matched tokens, while image scores are cosine similarities; the two scales therefore must not be compared directly.

9. Experimental Results

9.1. Test Set and Overall Results

The fixed evaluation set contains 12 labeled queries: five text queries, four voice transcripts, and three image queries. A query succeeds when its top-ranked product belongs to the declared relevant-product set. Eleven queries succeeded, producing an overall top-1 success rate of 91.67 percent.

Mode	Queries	Successful	Top-1 success rate
Text	5	4	80.00%
Voice	4	4	100.00%
Image	3	3	100.00%
Overall	12	11	91.67%

Table 12. Top-1 retrieval results on the fixed evaluation set

The only failed query was sneakers. The text method matches whole normalized tokens and does not currently expand sneakers to shoes, so it returned no candidate. This failure identifies a concrete vocabulary limitation rather than an unstable ranking result.

9.2. Image Model Comparison

A leave-one-out test measured whether the nearest different catalog photograph belonged to the same category. The exact query image was excluded from candidates. MobileNetV2 class logits achieved 29 of 40 correct categories. CLIP semantic embeddings achieved 38 of 40, an improvement of 22.5 percentage points. The stricter category-and-color result improved from 11 of 40 to 19 of 40.

Image representation	Correct category	Category accuracy	Correct category and color
MobileNetV2 ImageNet logits	29/40	72.5%	11/40 (27.5%)
CLIP ViT-B/32 embedding	38/40	95.0%	19/40 (47.5%)

Table 13. Leave-one-out image-similarity comparison

The category result shows that CLIP recognizes the product type more reliably. Color remains more difficult because backgrounds, lighting, clothing, and surrounding objects influence the visual embedding. The leave-one-out set is an internal regression proxy over 40 photographs and is not a claim of accuracy on unrestricted user images.

9.3. Demonstration Queries

Mode and input	Processing	Top result	Score
Text: black shoes	Bilingual normalization and whole-token matching	Metro Black Shoes	2.0000
Voice: find running shoes	Transcript validation, normalization, and text matching	Sprint Blue Running Shoes	2.0000
Image: query_a.png	CLIP 512-D embedding and cosine similarity	Metro Black Shoes	1.0000

Table 14. Required text, voice, and image demonstrations

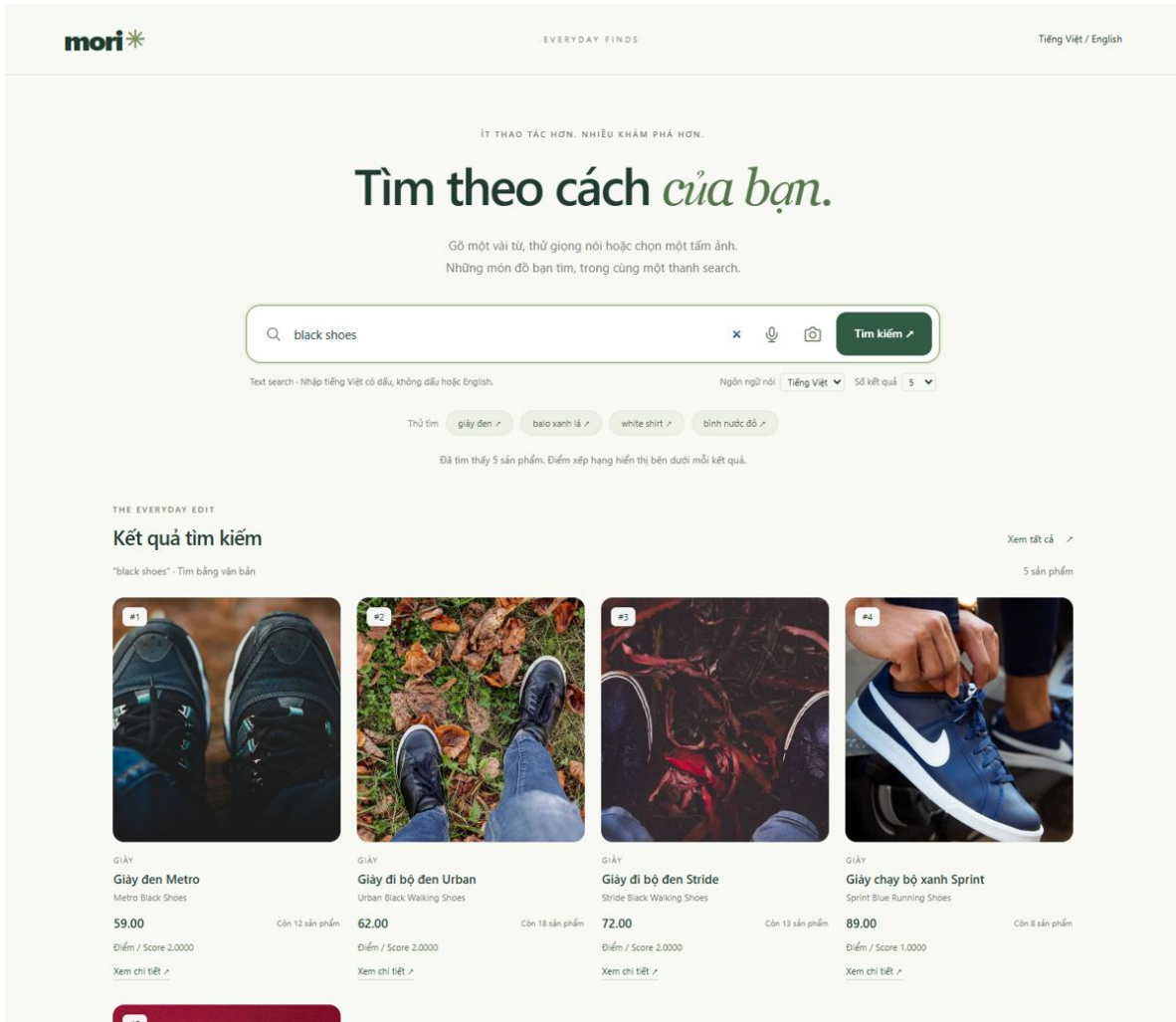


Figure 4. Web demonstration of the text query black shoes with ranked products and scores

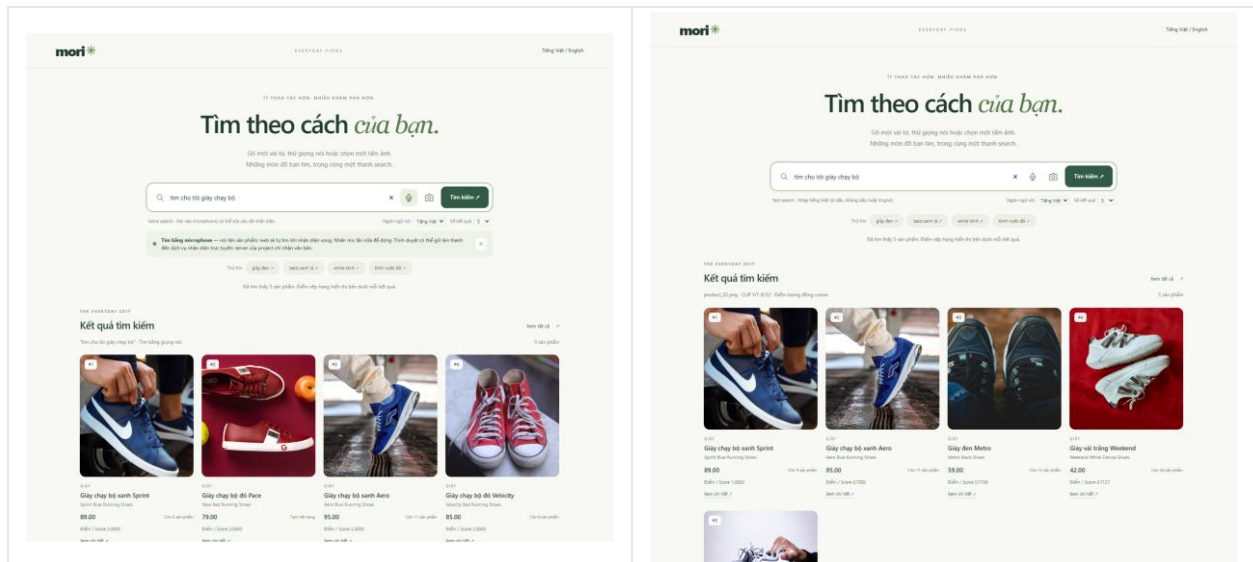


Figure 5. Voice search with microphone input and image search with CLIP-based ranking

9.4. Verification

The final verification run completed 36 automated tests successfully. The suite covers repository consistency, ranking, Vietnamese-English equivalence, Unicode normalization, microphone transcript flow, upload validation, CLIP embedding dimensions, catalog-image uniqueness, API boundaries, and leave-one-out image quality. A headless browser check also verified the 40-product catalog, text and voice results, image upload, diagnostic scores, product details, error states, 390-pixel mobile layout, and the absence of JavaScript errors.

10. Discussion

10.1. Strengths

The prototype maintains traceability from requirements and UML components to Python modules. All search modes produce the same query schema and ranked-product response, while their input-specific processing remains isolated behind services. The repository is balanced across four categories, photographs are unique and credited, and the web interface exposes the input, processing, returned products, and ranking score required by the assignment.

The implementation also completes several optional advanced features: microphone speech-to-text, real image embeddings, arbitrary image upload, bilingual search, and a responsive web interface. Replacing MobileNetV2 logits with CLIP changed only the image encoder and index construction, which demonstrates the extensibility intended by the three-layer design.

10.2. Limitations

- Text retrieval uses a finite Vietnamese-English dictionary and whole-token matching; it does not provide general translation, stemming, or semantic synonym expansion.
- Browser microphone recognition depends on browser support, permission, a secure localhost or HTTPS context, and potentially an external recognition service.
- CLIP has not been fine-tuned for this catalog. It recognizes product categories well, but color ranking remains sensitive to background, lighting, and the portion of the image occupied by the product.
- The evaluation contains only 12 functional queries and 40 leave-one-out image cases. It is suitable for regression testing but too small for a broad accuracy claim.
- Prices have no declared currency, order-related use cases remain design-only, and the prototype does not persist accounts, sessions, carts, or transactions.

10.3. Recommended Improvements

The next image-search improvement should separate product type from color. CLIP can first retrieve products of the correct category, after which foreground segmentation and a Lab or HSV color descriptor can re-rank candidates by product color. Each product should also have several photographs with different angles and backgrounds. Text search would benefit from synonym expansion for terms such as sneakers and a small semantic-text model. A larger external query set should measure category, color, and exact-product retrieval separately.

11. Conclusion

The project delivers a working multimodal e-commerce search prototype that follows the required three-layer architecture. ProductRepository manages a 40-item catalog; QueryService creates consistent queries; SearchService performs text or image retrieval; RankingService produces deterministic top-k results; and the CLI and web adapters demonstrate the complete workflow.

The required text, simulated voice, image-similarity, and ranking functions are implemented and demonstrated with explicit input, processing, returned products, and scores. The web version extends the baseline with actual microphone capture, arbitrary image upload, Vietnamese-English text normalization, real CLIP embeddings, responsive presentation, and diagnostics. Automated tests and browser checks confirm the implementation, while the reported failures and image-color results define the current limits without overstating performance.

11.1. References

OpenAI. CLIP: Connecting text and images. <https://github.com/openai/CLIP>

Xenova. CLIP ViT-B/32 ONNX model for Transformers.js. <https://huggingface.co/Xenova/clip-vit-base-patch32>

ONNX Model Zoo. MobileNet image classification models.

<https://github.com/onnx/models/tree/main/validated/vision/classification/mobilenet>

MDN Web Docs. SpeechRecognition API. <https://developer.mozilla.org/docs/Web/API/SpeechRecognition>